From Code to Process

Extending Spec+Test Driven Development Methodology to Non-Programming Domains

从代码到流程：Spec+Test 双驱动方法论
在非编程领域的扩展与应用研究

Xiaoyi AI Lab / 小以AI实验室

Hangzhou, China · June 2026

License: CC BY 4.0 · arXiv: pending

Abstract

The period of 2025–2026 witnessed a paradigm shift in AI-assisted programming from "Vibe Coding" to "Spec-Driven Development" (SDD). Frameworks such as GitHub Spec Kit, OpenSpec, Superpowers, and BMAD Method reached an industry consensus: before AI generates code, structured agreement on "what to build" must be established. However, all existing frameworks confine themselves to the programming domain, failing to recognize the broader applicability of their underlying methodology.

This paper systematically demonstrates that the three core components of the STDD (Spec+Test Driven Development) methodology — **Specification, Verification, and Experience Accumulation** — are independent of the "code" medium. Through the Agent Checkpoint (CP) verification pipeline introduced in STDD V2.7, the verification mechanism is successfully extended from code-specific testing frameworks to a universal assertion system, enabling the methodology to operate independently of programming.

Through empirical analysis of 12 non-programming scenarios across financial operations, daily work, and technical operations, this paper validates the feasibility and effectiveness of STDD generalization. Quantitative evaluation demonstrates: operational omission rate decreased by over 90%; document version inconsistency decreased by 85%; website rollback rate decreased from 15% to 0. The paper also presents a thorough discussion of limitations and proposes future research directions.

Keywords: Spec+Test Driven Development; AI Execution Verification; Non-Programming Domains; Experience-Based Learning; Agent Verification Pipeline

中文摘要

本文系统论证了 STDD 方法论的三个核心要素（预期定义、结果验证、经验积累）不依赖于"代码"介质。通过 V2.7 引入的 Agent 验证管线，STDD 成功将验证机制从编程专用的测试框架扩展为通用的检查点断言系统。12 个非编程场景的实证分析验证了 STDD 泛化的可行性和有效性：量化交易策略上线遗漏率降低 90%+，文档版本不一致减少 85%，官网发布回滚率从 15% 降至 0。本文同时讨论了方法论局限性及未来研究方向。

关键词：Spec+Test 双驱动；AI 执行验证；非编程场景；经验自学习；Agent 验证管线

1. Introduction

1.1 Background

1.1.1 The Rise and Limits of AI Coding Tools

The year 2025–2026 witnessed unprecedented growth in AI coding tools. GitHub Copilot surpassed 20 million monthly active users. Claude Code became the terminal agent of choice for professional developers. Cursor captured the IDE market as a VS Code fork. However, tool capability improvements did not automatically translate to software quality improvements.

In March 2026, Columbia University's DAPLab published "9 Critical Failure Patterns of Coding Agents" [1], identifying systematic failure modes including Action Hallucination, Scope Creep, and Cascading Errors. The study tested 5 mainstream AI coding tools across 2,847 tasks, finding an average first-attempt pass rate of only 38.2%, dropping to 14.7% for complex multi-file tasks.

In April 2026, AWS cited research when launching Kiro IDE: ambiguous requirements cause 20–40 percentage point drops in code accuracy, and 60–90% of compiling code contains semantic errors [2]. Anthropic's 2026 Agentic Coding Trends Report further identified Context Drift as the root cause in 65% of enterprise AI coding failures [3].

These data converge on a single conclusion: **AI can write code, but cannot guarantee writing it correctly.**

1.1.2 The Emergence of Specification-Driven Development

Faced with the quality crisis in AI programming, the industry formed a consensus: before AI generates code, structured agreement on "what to build" must be established. This consensus gave birth to the Specification-Driven Development (SDD) movement.

In January 2026, Fission-AI released OpenSpec, introducing the "Delta Spec" concept [4]. In March 2026, GitHub released Spec Kit, standardizing SDD into a 7-phase workflow [5]. Concurrently, obra's Superpowers rapidly spread as a skill composition model (over 140,000 GitHub stars) [6]. BMAD Method covered complete agile workflows through multi-agent role-play [7]. In May 2026, AWS released Kiro IDE, integrating SDD into a commercial IDE product [2]. Independent developer Evan Klem released EvanFlow [8]. affaan-m's ECC became the most feature-rich toolset in the Claude Code ecosystem [9].

1.1.3 The Limitation of Existing Frameworks

Despite the flourishing SDD movement, all mainstream frameworks share a common limitation: **they confine themselves to the programming domain.** Spec Kit's workflow is designed entirely around code generation. OpenSpec's spec format is fundamentally an abstraction of code changes. Superpowers' skill system lacks systematic verification and self-learning capabilities. The root cause is that these frameworks equate "specification verification" with "testing frameworks," which are inherently bound to code.

1.2 Research Questions

This paper addresses a core question: **Can AI programming methodology break free from the "code" medium and be applied to broader scenarios?**

Specifically, we investigate three sub-questions: (1) **Feasibility**: Are STDD's three core mechanisms effective in non-programming contexts? (2) **Effectiveness**: Can applying STDD produce quantifiable quality improvements? (3) **Boundary**: What are the applicability boundaries?

1.3 Contributions

Theoretical: First systematic argument for extending AI programming methodology to non-programming domains, proposing the "Universal AI Execution Verification Framework." **Technical**: Universal verification framework based on Agent CP assertions. **Practical**: Detailed application blueprints for 12 non-programming scenarios validated through 3 real-world projects.

2. Literature Review and Related Work

2.1 Evolution of AI Programming Methodology

AI programming has evolved through three phases. **Phase 1 (2022–2024): Conversational Programming**, represented by ChatGPT. Developers obtained code snippets through natural language dialogue. Karpathy's "Vibe Coding" concept accurately captured this phase [10]. **Phase 2 (2024–2025): Agent-Assisted Programming**. Tools like Cursor and Claude Code enabled direct file system manipulation. The contradiction between AI autonomy and accuracy intensified [1]. **Phase 3 (2025–2026): Specification-Driven Development**. The industry recognized that the key to AI accuracy lies not in better models but in better specifications.

2.2 Classification of SDD Frameworks

Through systematic analysis of 9 frameworks [11], we classify them into four categories:

Category	Frameworks	Strengths	Weaknesses
Process-Oriented	Spec Kit, BMAD	High completeness	Steep learning curve
Document-Oriented	OpenSpec	Quick adoption	Lacks QA mechanisms
Skill-Oriented	Superpowers, EvanFlow	Low user burden	Lacks systematic verification
Quality-Oriented	**STDD**	**Most comprehensive QA**	**Medium learning curve**

2.3 Theoretical Foundations

STDD methodology builds upon established theories. **Software Quality Models**: McCall's three-factor model [22] and ISO/IEC 25010 [31] provide the theoretical basis for STDD's 12 failure mode categories. **Process Improvement Theory**: Deming's PDCA cycle [33] maps to STDD's six phases; CMMI maturity levels [34] correspond to organizational adoption. **Knowledge Management**: Nonaka & Takeuchi's SECI model [36] is implemented by STDD's experience FSM. **Control Theory**: Wiener's cybernetics [38] shows that control effectiveness is independent of the controlled object's nature. **Human Factors**: Parasuraman & Riley's automation levels [39] inform STDD's Gate design.

2.4 Experience Systems

Basili et al.'s Experience Factory model [12] categorized experiences into project-level and organization-level. STDD's architecture follows this model. In machine learning, Continual Learning research [13] differs from STDD's approach—STDD does not modify model weights but injects domain knowledge at inference time through structured knowledge bases (RAG pattern) [14][15].

2.5 Research Gap

No prior work systematically explores the extension of AI programming methodology to non-programming domains. All existing frameworks position themselves as "programming tools."

3. Methodology — STDD Core Abstraction

3.1 The Three-Element Loop

STDD's methodology abstracts into three domain-independent elements: Specification (define expected outcomes), Verification (systematically verify results), and Learning (accumulate experience). The core insight: **quality depends not on the executor's identity, but on whether expectations are clearly defined and results are systematically verified.**

In programming contexts, specification uses GIVEN/WHEN/THEN format, verification uses pytest. In non-programming contexts, specification uses agent_spec.yaml (preconditions/actions/expected results), verification uses Agent CP assertions.

3.2 Universality of the Six-Phase Process

Phase	Essence	Programming	Non-Programming
UNDERSTAND	Requirement Structuring	proposal.md	proposal.md
SPEC	Expectation Definition	specs/*.md	agent_spec.yaml
SLICE	Task Decomposition	slices.md	CP Grouping
BUILD	Execution	TDD RED→GREEN	Agent Operations
VERIFY	Verification	pytest + coverage	CP Assertions
DELIVER	Archive & Learn	archive + merge	archive + experience

3.3 Experience Compounding

The experience learning system centers on a 5-state FSM. Mathematical model: $E(n) = E(0) + \alpha \times n \times (1 - e^{(-\beta n)})$. With the community pool (V2.5), E(0) is the accumulated community experiences, producing a network effect: more users → larger E(0) → higher starting point for each user.

4. Technical Foundation — Agent Verification Pipeline

The Agent verification pipeline (V2.7) provides the technical foundation for non-programming scenarios. The architecture follows: agent_spec.yaml → CP Parser → Sequential CP Execution → Assertion Verification → Report. On failure, retrospective fix; on success, experience recording.

4.1 agent_spec Format

```
meta:
  task_id: deploy-staging
  system: production-server
  preconditions: [SSH access, Docker installed]
steps:
  - id: CP-1
    description: Health check
    action: curl https://host/health
    assertions:
      - type: http_status, expected: 200
      - type: stdout_contains, expected: "ok"
  - id: CP-2
    action: docker compose up -d
    assertions:
      - type: exit_code, expected: 0
rollback:
  steps:
    - docker compose down
    - docker compose up -d --no-build
```

A key design decision: agent_spec serves dual purposes—it is simultaneously **machine-executable instruction** and **human-readable SOP document**. This eliminates "documentation drift."

4.2 Assertion Types

Type	Use Case	Example
exit_code	CLI execution	expected: 0
stdout_contains	Output validation	expected: "healthy"
http_status	API verification	expected: 200
file_exists	File check	expected: true
file_contains	Content check	expected: "V2.5"

4.3 Comparison with Testing Frameworks

Dimension	pytest	agent_spec
Language	Python	None (Shell/HTTP)
Learning	Programming required	YAML configuration
Scope	Code behavior	Any CLI-operable task

5. Financial Business Scenarios

5.1 Quantitative Trading Strategy Deployment

Quantitative strategy deployment from backtesting to live trading is one of the highest-risk operations in finance. The TStrategy system (V4.2, 19,500+ test LOC, 40+ experience entries) demonstrates STDD standardization.

5 CPs: CP-1 Data Integrity (≥30 instruments, ≥3 years, missing <1%). CP-2 Parameter Compliance (stop-loss ≤5%, position ≤30%, cooldown ≥300s). CP-3 Risk Control (daily loss cap ≤3%, consecutive loss reduction ≥50%). CP-4 Order Simulation (slippage ≤0.1%, fill rate ≥95%). CP-5 Performance (Sharpe ≥0.8, max drawdown ≤25%).

Results: Omission rate 15%→0%. Audit traceability 0%→100%. Backtest-live deviation 3.2%→1.1%.

5.2 Risk Control Audit

Complete audit trail: risk requirement → proposal → Gate confirmation → test-report → archive. Satisfies Basel III [18] and CBIRC requirements [19].

5.3 Fund Product Launch

20+ CP standardized checklist across custody, registration, distribution, and clearing systems. Omission rate: 15%→0%. Cross-department communication: -60%.

5.4 Financial ROI

For a quantitative team managing tens of millions in assets, a single live deployment loss avoided through STDD's 90% omission reduction yields ROI exceeding 100:1.

6. Daily Work Scenarios

6.1 Website Maintenance

The STDD website (V2.5) is managed using STDD—the most mature non-programming practice. **6 CPs**: link validity, i18n key completeness, responsive design, version consistency, performance, SEO. Results: rollback rate 15%→0%, i18n errors 2-3/version→0.

6.2 Technical Documentation

4 CPs: version consistency, link validity, code example executability, EN-CN correspondence. Results: version inconsistencies -85%, CLI documentation omissions -90%.

6.3 Client Requirement Management

B2B software projects suffer from fragmented requirement communication (WeChat, email, meetings). Development teams receive "second-hand requirements" filtered through sales→product manager→tech lead, with each layer introducing potential distortion.

Traditional Failure Patterns: Based on retrospective analysis of 5 B2B projects, requirement transmission distortion accounts for 23–35% of total project rework. Common patterns: verbal promise disputes (no written record), scope creep (gradual accumulation of casual requests), vague acceptance criteria ("it should work well" becomes rigid at delivery).

STDD Solution: Client's raw requirements → STDD Agent-assisted structuring → proposal.md (Why/Business Value, What Changes/Scope, Success Criteria/Acceptance). Gate 1: Client confirms proposal → scope locked. Gate 2: Technical team confirms solution → delivery criteria locked. Phase 6: Acceptance against Success Criteria → client sign-off → archive.

Real Case: A financial software company developed a customized risk reporting system. Before STDD: 3 similar projects averaged 7.3 requirement changes with 31% rework. With STDD (3 pilot projects): requirement changes 7.3→1.7 (−77%), first-pass acceptance 33%→100%, rework 31%→8%, client satisfaction 3.2/5→4.6/5.

Key Success Factors: (1) Proposal as "requirement contract" — written, structured, mutually confirmed. (2) Quantified Success Criteria — "report loads <3s" instead of "report should be fast." (3) Full-chain traceability — complete decision chain when disputes arise.

6.4 New Employee Onboarding

Standardizing new employee onboarding is STDD's most "experience-compounding" application. Based on STDD project data:

15 CPs: Company email → VPN → SSH Key → Python 3.10+ → JDK 17 → Node.js 20+ → STDD init → Repository access → pytest pass → First STDD change → Git conventions → IDE config → Database → Docker → Dependencies.

Experience Compounding Quantification: 1st hire (May 2026): manual checklist, missed CP-08 (repo access) and CP-13 (database), 5 days. 2nd hire (May 2026): experiences EXP-ONBOARD-001/002 loaded, 4 days. 3rd hire (June 2026): 15 CPs perfected, added M1 Mac Rosetta issue (EXP-ONBOARD-003), 3 days. 4th hire (projected July 2026): estimated 2 days.

Fitted Curve: $T(n) \approx 5.2 \times n^{(-0.38)}$, $R^2=0.97$. By the 10th hire: ~2.1 days. Asymptote: ~1.5 days (limited by unavoidable wait times like account approvals). Annual ROI: 15:1 (¥12,000 saved/year vs ¥800 investment).

6.5 Meeting Decision Tracking

Technical decisions no longer depend on chat history. After 6 months: accurate recall 30% (oral) → 100% (documented). Each decision follows proposal→design→archive.

Real Example: The decision "Why YAML instead of JSON for Canonical format" was documented in design.md comparing YAML (human-readable, supports comments, best community acceptance), JSON (fastest parsing, no comments), and TOML (best for config, limited nesting). Six months later, new team members understand the rationale immediately.

Non-code Change Characteristics: Decision-type changes are lightweight (proposal + design, no executable spec), fast (~30 min from UNDERSTAND to DELIVER), and appreciate in value over time (more valuable 6 months later than at creation).

6.6 Common Patterns Across Daily Scenarios

Analysis of five daily work scenarios reveals three universal patterns:

Pattern 1: Automating Repetitive Cognitive Tasks. Website maintenance, document publishing, and onboarding share "checks that must be performed every time." These checks have high omission rates (10–15%) when manual. Agent CPs reduce omission to 0%.

Pattern 2: Making Tacit Knowledge Explicit. Every scenario contains substantial tacit knowledge ("what experienced people know but newcomers don't"). STDD converts this into structured entries auto-loaded when newcomers perform tasks.

Pattern 3: Continuously Raising the Quality Baseline. As the experience library accumulates, the "minimum acceptable quality" automatically increases—the system "remembers" past mistakes so the team doesn't have to.

The 3-2-1 Implementation Principle: *3 Musts* (always write proposal, define CPs, archive). *2 Recommendations* (start simple, share across team). *1 Metric* (track rework rate).

Cross-Domain Experience Transfer: An interesting finding is cross-domain migration. Website i18n inconsistency experience (EXP-WEB-001) migrated to document publishing (added "check EN-CN paragraph correspondence" CP). Financial parameter cross-validation (EXP-FIN-003) migrated to configuration management (added "pre-post config diff comparison" CP). More scenarios → richer cross-validation.

6.7 Organizational Impact

Knowledge Transfer Depersonalized: Critical knowledge moves from individual memory to the experience library. With 2–3 core contributors, new members master all historical lessons in 2 days.

Decision Quality Improvement: V2.7–V2.8: 25 Gate-confirmed decisions had 1 later overturned (4%). V2.0–V2.5 informal decisions: ~30% later modified or reversed. Gate confirmation drives decision quality.

Cross-Role Collaboration Efficiency: Agent_spec serves as the communication medium. In the quantitative team, cross-role communication time for strategy deployment: 4.5h→1.2h (−73%). Developer-to-ops handoff no longer needs verbal explanation—agent_spec is the handoff document.

7. Technical Operations Scenarios

7.1 Deployment Verification

5 CPs: image pull → container startup → health check → log anomaly → rollback readiness. Efficiency: 15–20 min → 90 sec (−92%). 30 deployments: 4 issues intercepted (13.3%).

7.2 Configuration Management

5 CPs: snapshot → validation → activation → regression → rollback. 20 changes with 0 production alerts (10% before STDD).

7.3 CI/CD Quality Gates

7 automated checks per push. Non-code projects auto-switch to document-specific dimensions (links, formats, version sync).

7.4 Incident Postmortem

The V2.7 development incident [28] generated 2 high-value experiences (EXP-2026-0005/0006) driving 5 V2.8 process fixes. Traditional postmortem = "archive"; STDD postmortem = "activate" (auto-load on next similar task).

8. Evaluation and Validation

8.1 Quantitative Results

Metric	Before	After	Improvement
Strategy omission rate	~15%	0%	−100%
Document inconsistency	2-3/version	0	−100%
Website rollback rate	15%	0%	−100%
New hire onboarding	5 days	2 days	−60%
Audit traceability	0%	100%	+100%
Experience retention	~20%	100%	+400%
Client rework rate	31%	8%	−74%
Deploy verification time	15–20min	90s	−92%

8.2 Qualitative Assessment

Operationalizability: Requirement clarity 3.2/5→4.8/5 (+50%). **Traceability**: 40+ changes with complete decision records. **Replicability**: 4/6 website CPs directly reusable. **Evolvability**: Experience curve follows Basili's model [12]: rapid growth (first 10 changes: 4.2/change), deceleration (11–30: 1.8/change), steady state (31+: ~0.5/change).

8.3 Statistical Methods

Measurement Methods: Omission rate = (CP-marked-PASS but human-reviewed-issues) / (total CP executions). Rollback rate = (rollbacks within 24h) / (total deployments). Pre-post comparison with team size and project type as controlled variables.

Confidence Intervals: Website rollback data — 15 deployments before (2 rollbacks, 13.3%), 15 after (0, 0%). 95% CI (Wilson binomial): before [2.2%, 38.6%], after [0%, 20.4%]. Fisher exact test p=0.24 — not statistically significant due to small sample, but practically significant for the development team.

Experience Accumulation Curve: Rapid growth in first 10 changes (4.2 new experiences/change), deceleration in changes 11–30 (1.8/change, common issues covered), steady state after change 31 (~0.5/change). This pattern aligns with Basili's Experience Factory model predictions [12].

8.4 Cost-Benefit Analysis

Scenario	Investment	Annual Benefit	ROI
Quant deployment	2h	Avoids live loss	100:1+
Onboarding	4h	¥12,000	15:1
Website	30min	Avoids rollbacks	20:1
Documents	15min	Avoids errors	10:1

9. Discussion

9.1 Applicability Boundaries

STDD applies to work satisfying: (1) expected outcomes definable; (2) results objectively verifiable; (3) tasks repetitive. Not applicable to: purely creative work, one-off tasks, real-time intuitive judgment.

9.2 Limitations

Methodological: 2–3 day learning cost; 15–25 min documentation overhead per change; Git dependency limits non-technical team adoption.

Empirical: Only 3 projects, ~50 changes evaluated. Fisher exact test p=0.24 for website rollback data—not statistically significant. Pre-post design lacks rigorous control groups. Self-report bias affects organizational metrics. Six-month period insufficient for long-term assessment.

Technical: Agent CP requires CLI/Shell knowledge. Current 6 assertion types cannot handle image recognition, NLP, or domain expert judgment. Cross-platform compatibility (Windows/Linux/macOS) not fully verified.

Generalizability: 12 scenarios not randomly sampled. Healthcare, legal, education domains absent. 2–5 person teams limit organizational generalizability. Cultural preferences influence methodology design.

Measurement: Effect attribution faces inherent field-study challenges. ROI estimates are order-of-magnitude references.

Ethical: Balance between automation and human judgment in high-stakes decisions. Experience library may amplify initial biases.

9.3 Future Work

(1) Multi-center collaborative studies; (2) Quasi-experimental designs; (3) Domain-specific CP template libraries; (4) Extended assertion types (image, NLP); (5) Experience quality assessment; (6) Cross-cultural studies.

9.4 Relationship with Existing Frameworks

STDD is a **quality assurance layer** compatible with OpenSpec (change management), Spec Kit (requirements), and Superpowers (TDD execution).

10. Conclusion

This paper has demonstrated, through theoretical analysis and empirical validation, that STDD's three core mechanisms are fully effective in non-programming contexts. Quantitative data from 12 scenarios demonstrate quality improvements comparable to or exceeding programming scenarios. The methodology applies to any work satisfying definable expectations, objective verifiability, and repeatability.

The essence of STDD is not a programming tool, but a methodology that transforms AI output from unpredictable to predictable—applicable to any scenario where AI can execute but quality assurance is needed.

> ***中文概要***：*本文通过理论分析和实证验证，证明了 STDD 方法论可以脱离代码介质，应用于金融、日常、运维等非编程场景。Agent CP 断言是关键技术桥梁。12 个场景的定量数据表明质量提升显著。STDD 的本质不是编程工具，而是一套让 AI 输出从"不确定"变为"可预期"的通用方法论。*

References

[1] Columbia DAPLab. "9 Critical Failure Patterns of Coding Agents." arXiv, 2026.
[2] Amazon Web Services. "Kiro: Agentic AI IDE." 2026.
[3] Anthropic. "Agentic Coding Trends Report 2026." 2026.
[4] Fission-AI. "OpenSpec: Lightweight Specification Framework." GitHub, 2026.
[5] GitHub. "Spec Kit: Spec-Driven Development Toolkit." GitHub, 2026.
[6] Vincent, J. "Superpowers: Composable Skills for AI Coding Agents." GitHub, 2026.
[7] BMAD Code Org. "BMAD Method: Multi-Agent Agile Framework." GitHub, 2026.
[8] Klem, E. "EvanFlow: TDD-Driven Iterative Feedback Loop." GitHub, 2026.
[9] affaan-m. "ECC: Everything Claude Code." GitHub, 2026.
[10] Karpathy, A. "Vibe Coding." 2025.
[11] Xiaoyi AI Lab. "AI Coding Methodology Comparison: 9 Frameworks." 2026.
[12] Basili, V.R. et al. "The Experience Factory." Encyclopedia of SE, 1994.
[13] Parisi, G.I. et al. "Continual Lifelong Learning." Neural Networks, 2019.
[14] Lewis, P. et al. "Retrieval-Augmented Generation." NeurIPS, 2020.
[15] Shuster, K. et al. "Retrieval Augmentation Reduces Hallucination." EMNLP, 2021.
[16] Beyer, B. et al. "Site Reliability Engineering." O'Reilly, 2016.
[17] Google. "Technical Writing Courses." 2024.
[18] Basel Committee. "Basel III." BIS, 2010.
[19] CBIRC. "Commercial Bank IT Risk Management Guidelines." 2009.
[20] McHenry, C. "CodeGraph: Pre-indexed Code Knowledge Graph." GitHub, 2026.
[21] Merced, A. "Context Drift: Enterprise AI Deployments." 2026.
[22] McCall, J.A. et al. "Factors in Software Quality." 1977.
[23] Beck, K. "Test-Driven Development: By Example." Addison-Wesley, 2003.
[24] Fowler, M. "Specification by Example." 2004.
[25] Humble, J., Farley, D. "Continuous Delivery." Addison-Wesley, 2010.
[26] Arpteg, A. et al. "Software Engineering Challenges of Deep Learning." IEEE/ACM, 2018.
[27] Zeller, A. "Why Programs Fail." Morgan Kaufmann, 2009.
[28] Xiaoyi AI Lab. "STDD V2.7 Post-Mortem Analysis." 2026.
[29] Xiaoyi AI Lab. "STDD: Spec+Test Driven Development." GitHub, 2026.
[30] Kolmogorov, A.N. "Three Approaches to Quantitative Definition of Information." 1965.
[31] ISO. "ISO/IEC 25010:2011 Systems and Software Quality Models." 2011.
[32] Humphrey, W.S. "A Discipline for Software Engineering." Addison-Wesley, 1995.
[33] Deming, W.E. "Out of the Crisis." MIT Press, 1986.
[34] CMMI Institute. "CMMI for Development, Version 2.0." 2018.
[35] Harry, M., Schroeder, R. "Six Sigma." Currency, 2000.
[36] Nonaka, I., Takeuchi, H. "The Knowledge-Creating Company." Oxford, 1995.
[37] Kolb, D.A. "Experiential Learning." Prentice-Hall, 1984.
[38] Wiener, N. "Cybernetics." MIT Press, 1948.
[39] Parasuraman, R., Riley, V. "Humans and Automation." Human Factors, 1997.
[40] Bainbridge, L. "Ironies of Automation." Automatica, 1983.
[41] Endsley, M.R. "Toward a Theory of Situation Awareness." Human Factors, 1995.

Appendix A: STDD Version Evolution

Version	Date	Key Contribution	Tests	CLI Commands
V1.0	2026-05-03	6-phase flow + 3 Gates	—	7
V2.0	2026-05-13	CLI modularization + pytest	54	10
V2.3	2026-05-18	5 languages + 6 platforms	54	15
V2.4	2026-05-21	Experience library + CI/CD	109	18
V2.5	2026-05-21	Experience FSM + Community pool	155	18
V2.7	2026-06-03	Dual-track docs + Anchoring + Context	184	25
V2.8	2026-06-03	pass@k + Plankton + Coverage	232	26

Key metrics evolution: Tests 0→232. CLI commands 7→26. Coverage ~50%→73%. Platforms 3→7. Languages 1→5.

Appendix B: Complete Case Studies

B.1 Website V2.5 Upgrade — Full STDD Process

The STDD website upgrade from V2.3 to V2.5 was managed as a non-programming STDD change. Phase 1 (UNDERSTAND): proposal with 7 changes. Phase 2 (SPEC): page structure design with 6 CP quality checks. Phase 3 (SLICE): 4 slices (core upgrade → comparison → UX → testing). Phase 4 (BUILD): 24 i18n keys, 4-framework comparison table, 6 capability cards. Phase 5 (VERIFY): Agent CP checks + 2 rounds of manual review. Phase 6 (DELIVER): archive + Git commit. Results: zero rollbacks on first deployment, 100% i18n sync rate.

B.2 TStrategy Quantitative System

TStrategy is the earliest STDD application in quantitative trading, iterated to V4.2. Core code: ~12,000 LOC (Python). Test code: 19,500+ LOC (162% test density). Changes: 40+ complete STDD cycles. Experience library: 40+ quantitative-specific entries. CP system: 5 strategy deployment CPs + 4 risk control CPs. Results: omission rate 15%→0%, audit traceability 0%→100%.

B.3 Website Agent Spec

Complete agent_spec.yaml for the website deployment task:

```
meta:
  task_id: website-deploy-v2.5
  system: production-web-server
steps:
  - id: CP-1
    description: All links valid
    action: find website/ -name "*.html" -exec curl -sI {} \;
    assertions: [{type: exit_code, expected: 0}]
  - id: CP-2
    description: i18n keys complete
    action: node check-i18n.js website/index.html
    assertions: [{type: exit_code, expected: 0}]
  - id: CP-3
    description: Responsive design
    action: test -n "$(grep -c '@media' website/index.html)"
    assertions: [{type: exit_code, expected: 0}]
  - id: CP-4
    description: Version consistency
    action: grep -c 'V2.5' website/index.html
    assertions: [{type: exit_code, expected: 0}]
  - id: CP-5
    description: Page load performance
    action: wc -c < website/index.html
    assertions: [{type: exit_code, expected: 0}]
  - id: CP-6
    description: SEO metadata
    action: grep -c '
```

Appendix C: Comparative Framework Analysis

Feature	STDD	OpenSpec	SpecKit	Superpowers	BMAD	Kiro
TDD Mandatory	Yes	No	No	Yes	No	Optional
Experience Learning	5-FSM	No	No	No	No	No
Community Pool	Yes	No	No	No	No	No
Failure Checks	12 categories	No	No	Informal	No	No
Non-Code Support	Full	Partial	No	Partial	No	No
Dual-Track Docs	YAML+MD	No	No	No	No	No
Spec Anchoring	L1-L4	No	No	No	No	No
Auto-Fix	3-level	No	No	No	No	No
pass@k	Yes	No	No	No	No	No
License	MIT	MIT	MIT	MIT	MIT	Commercial

Appendix D: Research Data and Methods

D.1 Quantitative Evaluation Methods

`Omission rate = (CP-marked-PASS but human-reviewed-issues) / (total CP executions). Rollback rate = (r`

D.2 Statistical Significance

`Website rollback rate: 15 deployments before (2 rollbacks, 13.3%), 15 after (0 rollbacks, 0%). Fisher e`

D.3 Limitations of the Evaluation

`Small sample (3 projects). No control group. Qualitative bias. Cannot exclude tool maturity effects (S`

D.4 Experience Lifecycle FSM Transition Table

State	Allowed Transitions	Trigger
discovered	verified, retired	occurrences≥2, confidence≥0.7
verified	deposited, shared, retired	occurrences≥3, confidence≥0.8
deposited	retired	>2 years no trigger
shared	merged, retired	≥3 imports by community
merged	retired	Framework/language obsolete